Matplotlib — Class Notes for Fresher Data Analysts

1. Introduction to Matplotlib

Matplotlib is a Python library used to create charts and visualizations from data.

For a Data Analyst, Matplotlib is useful for:

- Understanding patterns and trends
- Comparing categories
- Identifying distributions
- Finding relationships between variables
- Detecting unusual values
- Creating charts for reports and presentations

The most commonly used interface is `matplotlib.pyplot`.

```
import matplotlib.pyplot as plt
```

A typical Matplotlib workflow is:

```
import matplotlib.pyplot as plt

plt.plot(x, y)
plt.title("Sales Trend")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()
```

Sample data used throughout the notes

```
months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
sales = [12000, 15000, 13500, 18000, 21000, 19500]

products = ["Laptop", "Mobile", "Tablet", "Monitor"]
revenue = [50000, 75000, 40000, 30000]
```

---

2. Line Charts

A **line chart** is primarily used to show **trends or changes over an ordered variable**, especially time.

Typical Data Analyst use cases:

- Monthly sales
- Daily website traffic
- Yearly revenue
- Temperature trends
- Customer growth
- Expenses over time

Basic Line Chart

```
import matplotlib.pyplot as plt

months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
sales = [12000, 15000, 13500, 18000, 21000, 19500]

plt.plot(months, sales)

plt.show()
```

Adding a Title and Axis Labels

```
plt.plot(months, sales)

plt.title("Monthly Sales")
plt.xlabel("Month")
plt.ylabel("Sales")

plt.show()
```

Adding Markers

Markers make individual observations easier to identify.

```
plt.plot(
    months,
    sales,
    marker="o"
)

plt.title("Monthly Sales")
plt.xlabel("Month")
plt.ylabel("Sales")

plt.show()
```

Common markers:

Marker	Meaning
`o`	Circle
`s`	Square
`^`	Triangle
`D`	Diamond
`*`	Star
`x`	X

Example:

```
plt.plot(months, sales, marker="o")
```

Customizing Line Color

```
plt.plot(
    months,
    sales,
    color="blue"
)
```

You can also use common colors such as:

```
"red"
"green"
```

```
"orange"
"purple"
"black"
"gray"
```

Hexadecimal colors are useful for professional reports:

```
plt.plot(
    months,
    sales,
    color="#1f77b4"
)
```

Changing Line Width

```
plt.plot(
    months,
    sales,
    linewidth=2
)
```

Short form:

```
plt.plot(months, sales, lw=2)
```

Changing Line Style

```
plt.plot(
    months,
    sales,
    linestyle="--"
)
```

Common styles:

```
-     Solid
--    Dashed
:     Dotted
-.    Dash-dot
```

Example:

```
plt.plot(months, sales, linestyle="--")
```

Combining Customizations

```
plt.plot(
    months,
    sales,
    color="#1f77b4",
    marker="o",
    linestyle="-",
    linewidth=2,
    markersize=6
)

plt.title("Monthly Sales")
plt.xlabel("Month")
plt.ylabel("Sales")

plt.show()
```

Multiple Lines

Suppose we have sales for two years:

```
sales_2025 = [12000, 15000, 13500, 18000, 21000, 19500]
sales_2026 = [14000, 16000, 15000, 20000, 23000, 22000]

plt.plot(months, sales_2025, marker="o", label="2025")
plt.plot(months, sales_2026, marker="o", label="2026")

plt.title("Monthly Sales Comparison")
plt.xlabel("Month")
plt.ylabel("Sales")

plt.legend()

plt.show()
```

Why `label` and `legend()` matter

When multiple lines appear in a chart, the viewer needs to know what each line represents.

```
label="2025"
```

creates the label, while:

```
plt.legend()
```

displays it.

Grid

A light grid can make values easier to read.

```
plt.plot(months, sales, marker="o")

plt.grid(
    axis="y",
    linestyle="--",
    alpha=0.4
)

plt.show()
```

For analytical reports, a subtle grid is usually better than a heavy grid.

---

3. Bar Charts

A **bar chart** is used primarily to compare values across categories.

Common use cases:

- Sales by product
- Revenue by region
- Employees by department
- Orders by category
- Profit by business segment

Basic Bar Chart

```
products = ["Laptop", "Mobile", "Tablet", "Monitor"]
revenue = [50000, 75000, 40000, 30000]

plt.bar(products, revenue)
```

```
plt.title("Revenue by Product")
plt.xlabel("Product")
plt.ylabel("Revenue")

plt.show()
```

Changing Bar Color

```
plt.bar(
    products,
    revenue,
    color="steelblue"
)
```

Different Colors for Each Bar

```
colors = ["#4C78A8", "#F58518", "#54A24B", "#E45756"]

plt.bar(
    products,
    revenue,
    color=colors
)

plt.show()
```

Use different colors only when there is a meaningful reason. For simple category comparison, one consistent color often looks cleaner.

Changing Bar Width

```
plt.bar(
    products,
    revenue,
    width=0.6
)
```

The default bar width is usually sufficient, but adjusting it can improve readability.

Horizontal Bar Chart

Use `barh()` when category names are long.

```
plt.barh(
    products,
    revenue
)

plt.title("Revenue by Product")
plt.xlabel("Revenue")
plt.ylabel("Product")

plt.show()
```

For example, long department names are often easier to read horizontally.

Sorting Bars

Sorted bars make comparisons easier.

```
products = ["Laptop", "Mobile", "Tablet", "Monitor"]
revenue = [50000, 75000, 40000, 30000]

sorted_data = sorted(
    zip(products, revenue),
    key=lambda x: x[1],
    reverse=True
)

products_sorted = [x[0] for x in sorted_data]
revenue_sorted = [x[1] for x in sorted_data]

plt.bar(products_sorted, revenue_sorted)

plt.title("Revenue by Product")
plt.ylabel("Revenue")

plt.show()
```

Adding Data Labels

Data labels are useful when exact values matter.

```
bars = plt.bar(products, revenue)

for bar in bars:
    value = bar.get_height()

    plt.text(
        bar.get_x() + bar.get_width() / 2,
        value,
        f"{value:,}",
        ha="center",
        va="bottom"
    )

plt.title("Revenue by Product")
plt.ylabel("Revenue")

plt.show()
```

Why use data labels?

A chart should help the viewer understand the data quickly. Labels can be useful when there are only a few bars and exact values matter.

Avoid labeling every bar when the chart contains dozens of categories.

---

4. Histograms

A **histogram** shows the **distribution of a numerical variable**.

Unlike a bar chart, a histogram deals with continuous numerical data and groups values into intervals called **bins**.

Typical uses:

- Salary distribution
- Age distribution

- Order value distribution
- Delivery time distribution
- Exam score distribution

Basic Histogram

```
ages = [22, 25, 28, 31, 24, 35, 29, 32, 27, 26,
        30, 33, 24, 28, 29, 31, 36, 25, 27, 30]

plt.hist(ages)

plt.title("Age Distribution")
plt.xlabel("Age")
plt.ylabel("Frequency")

plt.show()
```

Understanding Bins

Bins determine how the values are grouped.

```
plt.hist(
    ages,
    bins=5
)
```

Try different values:

```
bins=5
bins=10
bins=15
```

The choice of bins can significantly affect the appearance of the distribution.

Customizing Histogram Color

```
plt.hist(
    ages,
    bins=8,
    color="steelblue",
    edgecolor="black"
```

```
)

plt.show()
```

The `edgecolor` helps distinguish adjacent bins.

Density Histogram

Instead of showing frequency, you can show density:

```
plt.hist(
    ages,
    bins=8,
    density=True,
    color="steelblue",
    edgecolor="black"
)

plt.show()
```

This is useful when comparing distributions with different sample sizes.

Important Interpretation Questions

When looking at a histogram, ask:

- Where are most observations concentrated?
- Is the distribution symmetric?
- Is it skewed?
- Are there multiple peaks?
- Are there unusually high or low values?
- How wide is the distribution?

---

5. Scatter Plots

A **scatter plot** is used to examine the relationship between **two numerical variables**.

Examples:

- Advertising spend vs sales

- Experience vs salary
- Age vs income
- Temperature vs electricity consumption
- Study hours vs marks

Basic Scatter Plot

```
advertising = [10, 20, 30, 40, 50, 60, 70]
sales = [25, 30, 35, 42, 45, 55, 62]

plt.scatter(
    advertising,
    sales
)

plt.title("Advertising Spend vs Sales")
plt.xlabel("Advertising Spend")
plt.ylabel("Sales")

plt.show()
```

Customizing Points

```
plt.scatter(
    advertising,
    sales,
    color="steelblue",
    s=80,
    alpha=0.7
)

plt.show()
```

Important parameters:

- `color` → point color
- `s` → point size
- `alpha` → transparency

Why Use Transparency?

When many observations overlap, points can hide one another.

`alpha=0.5`

makes overlapping observations easier to see.

Different Colors

```
plt.scatter(
    advertising,
    sales,
    color="#2E86AB",
    s=70,
    alpha=0.7
)
```

Adding a Third Variable Through Color

Suppose we have profit:

```
profit = [5, 8, 7, 10, 12, 15, 18]
```

We can use profit to control color:

```
plt.scatter(
    advertising,
    sales,
    c=profit,
    cmap="viridis",
    s=80
)

plt.colorbar(label="Profit")

plt.xlabel("Advertising Spend")
plt.ylabel("Sales")
plt.title("Advertising Spend vs Sales")

plt.show()
```

This allows a scatter plot to communicate three variables:

- X-axis → advertising

- Y-axis → sales
- Color → profit

Common Scatter Plot Interpretations

Positive relationship

As X increases, Y generally increases.

Negative relationship

As X increases, Y generally decreases.

No obvious relationship

There is no clear pattern between the variables.

Remember: **a scatter plot showing a relationship does not by itself establish causation.**

---

6. Pie Charts

A **pie chart** shows how categories contribute to a whole.

Example:

- Market share
- Expense breakdown
- Sales contribution
- Customer segment composition

Pie charts work best when there are **few categories**.

Basic Pie Chart

```
categories = ["Electronics", "Clothing", "Furniture", "Books"]
sales = [40, 25, 20, 15]

plt.pie(
    sales,
```

```
    labels=categories
)

plt.title("Sales Distribution")

plt.show()
```

Showing Percentages

```
plt.pie(
    sales,
    labels=categories,
    autopct="%1.1f%%"
)

plt.title("Sales Distribution")

plt.show()
```

`autopct` controls the percentage display.

For example:

```
autopct="%1.0f%%"
```

displays whole percentages.

Custom Colors

```
colors = [
    "#4C78A8",
    "#F58518",
    "#54A24B",
    "#E45756"
]

plt.pie(
    sales,
    labels=categories,
    autopct="%1.1f%%",
    colors=colors
```

```
)

plt.show()
```

Exploding a Slice

```
explode = [0.05, 0, 0, 0]

plt.pie(
    sales,
    labels=categories,
    autopct="%1.1f%%",
    explode=explode
)

plt.show()
```

`explode` separates selected slices slightly from the center.

Start Angle

```
plt.pie(
    sales,
    labels=categories,
    autopct="%1.1f%%",
    startangle=90
)

plt.show()
```

Keeping the Pie Circular

```
plt.axis("equal")
```

or, when working with an axes object:

```
ax.set_aspect("equal")
```

This prevents the pie from appearing stretched.

When to Avoid Pie Charts

Avoid pie charts when:

- There are many categories.
- Values are very similar.
- Exact comparisons are important.
- You need to compare several groups.

A bar chart is often easier to compare precisely.

---

7. Box Plots

A **box plot** summarizes the distribution of numerical data.

It is particularly useful for:

- Comparing distributions
- Identifying spread
- Finding potential outliers
- Comparing salaries between departments
- Comparing delivery times across regions

A box plot is based on:

- Minimum/non-outlier lower value
- First quartile — Q1
- Median — Q2
- Third quartile — Q3
- Maximum/non-outlier upper value
- Potential outliers

Basic Box Plot

```
salaries = [
    30000, 35000, 40000, 42000, 45000,
    47000, 50000, 52000, 55000, 60000
]

plt.boxplot(salaries)
```

```
plt.title("Salary Distribution")
plt.ylabel("Salary")

plt.show()
```

Understanding the Box

The box represents the **interquartile range (IQR)**:

```
IQR = Q3 - Q1
```

The line inside the box represents the **median**.

Points outside the whiskers may be shown as potential outliers according to the box plot's outlier rule.

Multiple Box Plots

Suppose we have salaries for three departments:

```
sales = [30000, 35000, 40000, 42000, 45000]
hr = [32000, 36000, 39000, 41000, 44000]
it = [45000, 50000, 55000, 60000, 65000]
```

```
plt.boxplot(
    [sales, hr, it],
    labels=["Sales", "HR", "IT"]
)

plt.title("Salary Distribution by Department")
plt.ylabel("Salary")

plt.show()
```

This makes distribution comparisons easy.

Horizontal Box Plot

```
plt.boxplot(
    salaries,
    vert=False
)
```

```
plt.xlabel("Salary")
plt.title("Salary Distribution")

plt.show()
```

Horizontal box plots are useful when category names are long.

---

8. Subplots

Subplots allow multiple charts to be displayed within one figure.

They are useful when you want to analyze related information together.

For example:

- Sales trend
- Revenue by product
- Sales distribution
- Profit vs advertising

Basic Subplots

```
fig, axes = plt.subplots(2, 2)

axes[0, 0].plot(months, sales)
axes[0, 0].set_title("Sales Trend")

axes[0, 1].bar(products, revenue)
axes[0, 1].set_title("Revenue by Product")

axes[1, 0].hist(ages, bins=8)
axes[1, 0].set_title("Age Distribution")

axes[1, 1].scatter(advertising, sales)
axes[1, 1].set_title("Advertising vs Sales")

plt.show()
```

This creates a **2 × 2 grid**.

Understanding `fig` and `axes`

```
fig, axes = plt.subplots(2, 2)
```

- `fig` represents the entire figure.
- `axes` contains the individual plotting areas.

For a 2 × 2 layout:

```
axes[0,0]    axes[0,1]

axes[1,0]    axes[1,1]
```

One Row, Multiple Charts

```
fig, axes = plt.subplots(1, 3)

axes[0].plot(months, sales)
axes[1].bar(products, revenue)
axes[2].hist(ages)

plt.show()
```

One Column, Multiple Charts

```
fig, axes = plt.subplots(3, 1)

axes[0].plot(months, sales)
axes[1].bar(products, revenue)
axes[2].hist(ages)

plt.show()
```

Sharing an Axis

Sharing axes is useful when charts need to use the same scale.

```
fig, axes = plt.subplots(
    2,
    1,
```

```
    sharex=True
)

axes[0].plot(months, sales)
axes[1].plot(months, sales_2026)

plt.show()
```

Figure Size

```
fig, axes = plt.subplots(
    2,
    2,
    figsize=(12, 8)
)
```

`figsize` is specified as:

```
(width, height)
```

in inches.

---

9. Customized Publication/Report-Quality Figures

Creating a chart is only part of data visualization.

A professional figure should be:

- Easy to read
- Properly labeled
- Visually consistent
- Appropriately sized
- Free from unnecessary decoration
- Suitable for its intended medium

Figure Size

```
plt.figure(figsize=(10, 6))
```

For reports, larger figures may be useful.

```
fig, ax = plt.subplots(figsize=(10, 6))
```

Title Customization

```
plt.title(
    "Monthly Sales Performance",
    fontsize=16,
    fontweight="bold"
)
```

With an axes object:

```
ax.set_title(
    "Monthly Sales Performance",
    fontsize=16,
    fontweight="bold"
)
```

Axis Labels

```
plt.xlabel(
    "Month",
    fontsize=12
)

plt.ylabel(
    "Sales ($)",
    fontsize=12
)
```

Tick Label Customization

```
plt.xticks(
    fontsize=10
)

plt.yticks(
    fontsize=10
)
```

With axes:

```
ax.tick_params(
    axis="both",
    labelsize=10
)
```

Rotating Labels

Long category names can overlap.

```
plt.xticks(rotation=45)
```

For example:

```
plt.xticks(
    rotation=45,
    ha="right"
)
```

Grid Customization

Instead of a heavy grid:

```
plt.grid(
    axis="y",
    linestyle="--",
    alpha=0.3
)
```

A subtle grid generally improves readability without dominating the chart.

Removing Unnecessary Spines

Spines are the lines around the plotting area.

With an axes object:

```
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
```

This produces a cleaner appearance.

You can also remove the left or bottom spine when appropriate:

```
ax.spines["left"].set_visible(False)
```

Color Consistency

For professional reports, use a consistent color palette.

```
primary_color = "#2F5597"
secondary_color = "#ED7D31"
```

Then reuse the colors:

```
plt.plot(
    months,
    sales,
    color=primary_color
)
```

Consistent colors make multiple charts look like they belong to the same report.

---

10. Annotations

Annotations are useful when a specific data point deserves attention.

For example, suppose June has the highest sales.

```
months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
sales = [12000, 15000, 13500, 18000, 21000, 19500]

fig, ax = plt.subplots(figsize=(10, 6))

ax.plot(
    months,
    sales,
    marker="o",
    color="#2F5597"
)

ax.annotate(
    "Peak sales",
```

```
    xy=("May", 21000),
    xytext=("Apr", 23000),
    arrowprops=dict(
        arrowstyle="->"
    )
)

ax.set_title("Monthly Sales")
ax.set_xlabel("Month")
ax.set_ylabel("Sales")

plt.show()
```

Annotations should be used selectively.

Too many annotations make a chart difficult to read.

---

11. Formatting Numbers

For reports, displaying raw numbers like:

```
1500000
```

is less readable than:

```
1,500,000
```

Matplotlib supports formatting through `matplotlib.ticker`.

```
from matplotlib.ticker import StrMethodFormatter

fig, ax = plt.subplots(figsize=(10, 6))

ax.bar(products, revenue)

ax.yaxis.set_major_formatter(
    StrMethodFormatter("{x:,.0f}")
)

ax.set_title("Revenue by Product")
ax.set_ylabel("Revenue")
```

```
plt.show()
```

For percentages, percentage formatting can also be applied to axes when appropriate.

---

12. `tight_layout()`

Sometimes labels or titles overlap.

Use:

```
plt.tight_layout()
```

Example:

```
plt.figure(figsize=(10, 6))

plt.plot(months, sales)

plt.title("Monthly Sales")
plt.xlabel("Month")
plt.ylabel("Sales")

plt.tight_layout()

plt.show()
```

For subplots:

```
fig, axes = plt.subplots(2, 2, figsize=(12, 8))

# charts...

plt.tight_layout()
plt.show()
```

This is one of the simplest ways to improve figure spacing.

---

13. Saving Publication/Report Figures

Instead of only displaying the chart:

```
plt.show()
```

you can save it:

```
plt.savefig(
    "monthly_sales.png",
    dpi=300,
    bbox_inches="tight"
)
```

Important parameters

`dpi=300`

is commonly useful for high-resolution output.

`bbox_inches="tight"`

helps prevent labels and other content from being unnecessarily cut off.

Saving as PDF

```
plt.savefig(
    "monthly_sales.pdf",
    bbox_inches="tight"
)
```

Vector formats such as PDF can be particularly useful for documents where scalability matters.

---

14. A Complete Report-Quality Example

Here is a complete example combining several important customization techniques.

```
import matplotlib.pyplot as plt
from matplotlib.ticker import StrMethodFormatter

months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
sales = [12000, 15000, 13500, 18000, 21000, 19500]
```

```
fig, ax = plt.subplots(figsize=(10, 6))

ax.plot(
    months,
    sales,
    color="#2F5597",
    marker="o",
    linewidth=2.5,
    markersize=7
)

ax.set_title(
    "Monthly Sales Performance",
    fontsize=16,
    fontweight="bold"
)

ax.set_xlabel(
    "Month",
    fontsize=11
)

ax.set_ylabel(
    "Sales",
    fontsize=11
)

ax.yaxis.set_major_formatter(
    StrMethodFormatter("{x:,.0f}")
)

ax.grid(
    axis="y",
    linestyle="--",
    alpha=0.3
)

ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)

ax.annotate(
```

```
    "Peak sales",
    xy=("May", 21000),
    xytext=("Apr", 22500),
    arrowprops=dict(
        arrowstyle="->",
        color="black"
    )
)

plt.tight_layout()

plt.savefig(
    "monthly_sales.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```

This example demonstrates several skills expected from a beginner Data Analyst:

- Creating a chart
- Choosing an appropriate chart type
- Adding a title
- Labeling axes
- Formatting numbers
- Customizing colors
- Customizing markers
- Adjusting line width
- Adding a grid
- Removing unnecessary spines
- Annotating an important observation
- Controlling figure size
- Saving a high-resolution figure

---

15. Figure vs Axes — Important Concept

Beginners often confuse:

```
plt
```

with:

```
fig, ax = plt.subplots()
```

For simple charts, this is perfectly fine:

```
plt.plot(months, sales)
plt.title("Sales")
plt.show()
```

For professional or multi-chart work, the **object-oriented approach** is generally more flexible:

```
fig, ax = plt.subplots()

ax.plot(months, sales)
ax.set_title("Sales")
ax.set_xlabel("Month")
ax.set_ylabel("Revenue")

plt.show()
```

Think of it this way

```
Figure
└── Axes
    ├── Plot
    ├── Title
    ├── X-axis
    ├── Y-axis
    └── Other visual elements
```

When working with subplots, the object-oriented approach becomes especially useful.

---

16. Chart Selection for Data Analysts

Choosing the right chart is as important as knowing the syntax.

Analytical Question	Recommended Chart
How has something changed over time?	Line chart
Which category is larger?	Bar chart

Analytical Question	Recommended Chart
How are numerical values distributed?	Histogram
Are two numerical variables related?	Scatter plot
How does a whole divide among a few categories?	Pie chart
How do distributions compare and where are potential outliers?	Box plot
How can several related views be shown together?	Subplots

---

17. Essential Customization Cheat Sheet

Line chart

```
plt.plot(
    x,
    y,
    color="blue",
    marker="o",
    linestyle="-",
    linewidth=2
)
```

Bar chart

```
plt.bar(
    categories,
    values,
    color="steelblue",
    width=0.7
)
```

Histogram

```
plt.hist(
    values,
    bins=10,
    color="steelblue",
```

```
    edgecolor="black"
)
```

Scatter plot

```
plt.scatter(
    x,
    y,
    color="steelblue",
    s=60,
    alpha=0.7
)
```

Pie chart

```
plt.pie(
    values,
    labels=labels,
    autopct="%1.1f%%",
    startangle=90
)
```

Box plot

```
plt.boxplot(
    data,
    labels=labels
)
```

Subplots

```
fig, axes = plt.subplots(
    2,
    2,
    figsize=(12, 8)
)
```

Professional finishing

```
plt.tight_layout()
```

```
plt.savefig(
    "figure.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```

---

18. Fresher Data Analyst Checklist

Before considering a Matplotlib visualization complete, a fresher should be able to answer:

Line charts

- Can I create a basic line chart?
- Can I add markers?
- Can I plot multiple lines?
- Can I add a legend?
- Can I customize line color, style and width?

Bar charts

- Can I create vertical and horizontal bars?
- Can I change colors and widths?
- Can I add data labels?
- Can I sort categories before plotting?

Histograms

- Do I understand bins?
- Can I change the number of bins?
- Can I interpret the shape of a distribution?
- Can I identify potential unusual observations?

Scatter plots

- Can I visualize two numerical variables?
- Can I change point size and transparency?
- Can I use color to represent another variable?

- Can I interpret the direction and strength of a visible relationship without confusing correlation with causation?

Pie charts

- Can I display percentages?
- Can I customize colors?
- Can I highlight a slice?
- Do I know when a pie chart is inappropriate?

Box plots

- Can I create a basic box plot?
- Can I compare multiple groups?
- Do I understand median, quartiles and IQR?
- Can I identify potential outliers?

Subplots

- Can I create a grid of charts?
- Can I control rows and columns?
- Can I set figure size?
- Can I share axes where appropriate?

Publication/report-quality figures

- Can I control figure size?
- Can I create readable titles and labels?
- Can I format tick labels?
- Can I use consistent colors?
- Can I control gridlines and spines?
- Can I add useful annotations?
- Can I format numbers?
- Can I use `tight_layout()` ?
- Can I save a high-resolution figure?

Final takeaway

For a fresher Data Analyst, the goal is **not to memorize every Matplotlib parameter**. The important skill is being able to take a business question, choose an appropriate chart, create it, and customize it

enough that the result is **accurate, readable, and presentation-ready**.

The core workflow to remember is:

```
Understand the question
        ↓
Choose the appropriate chart
        ↓
Plot the data
        ↓
Add meaningful title and labels
        ↓
Improve readability
        ↓
Highlight important insights
        ↓
Check for misleading presentation
        ↓
Save/share the final figure
```